P-System: Deterministic Event Processing with Causal Ordering, Canonical Event Identity, and Explicit Conflict Handling

Author: Paolo Amodio

Version: 0.1.0

Research artifact commit: `ee8b054e5f3316e537a2f9a62b6d16fe18ada6cf`

Release tag: `v0.1.0`

Date: 19 October 2026

Status: Research preprint — preliminary results

Abstract

Distributed systems must process events despite partial knowledge, message duplication, concurrent activity, network interruptions, and conflicting histories. This paper presents P-System, an experimental software project investigating deterministic event processing through canonical event identity, causal dependency analysis, explicit conflict handling, and deterministic local state projection.

P-System combines a Python implementation with a separate Lean 4 formal model. The reported development baseline consists of 440 passing Python tests and a successful Lean build comprising 31 build jobs. Project documentation records 16 principal formal modules (P01–P16) and 89 theorem declarations. These results provide evidence of executable test coverage and machine-checked formalization. They do not establish complete correspondence between the Python implementation and the formal model, universal distributed-system correctness, or a performance advantage over existing approaches. No comparative benchmark results are claimed.

The paper describes the architecture, setup and execution commands, verification methodology, available evidence, limitations, and future research directions. Its contribution is an experimental research artifact with an explicit separation between formal claims, empirical tests, environmental assumptions, and unresolved verification obligations.

Keywords: distributed systems; event processing; causal ordering; canonical event identity; conflict resolution; deterministic execution; formal verification; Lean 4.

1. Introduction

Distributed systems may lack a globally consistent view of events. Nodes can receive messages in different orders, encounter duplicate deliveries, remain disconnected, or observe incompatible histories. A robust event-processing architecture must distinguish causal dependencies from concurrent activity and define how conflicts affect local state.

Lamport's logical-clock work provides a foundational account of causal ordering [1]. Conflict-free replicated data types provide a framework for convergence under defined replication and update conditions [2]. Lean 4 supports machine-checked reasoning about formal propositions [3].

P-System investigates these concerns through an experimental Python implementation and a separate Lean 4 model. It does not claim that every system property has been established or that the approach outperforms existing distributed systems.

1.1 Research questions

- **RQ1 — Determinism:** Under what conditions does event processing produce a deterministic state transition?

- **RQ2 — Causality:** How can causal dependencies constrain event ordering and execution?
- **RQ3 — Conflicts:** How should incompatible event histories be represented without arbitrarily selecting a winner?
- **RQ4 — Verification:** Which properties can be established formally, and which remain dependent on implementation tests or assumptions?
- **RQ5 — Evaluation:** What experiments are needed to establish performance, scalability, and operational limits?

2. System Overview

P-System is an experimental distributed event-processing project implemented in Python and accompanied by a separate Lean 4 formal model.

Component	Responsibility
`core/`	Event processing, state derivation, dependencies, conflicts, execution planning
`network/`	Nodes, peer communication, synchronization, reconciliation, reconnection, secure transport
`crypto/`	Identity and signature-related functionality
`tests/`	Automated component and integration tests
`benchmarks/`	Scripts intended for performance and scalability experiments
`formal/lean/`	Formal definitions and machine-checked proofs
`docs/`	Specifications, proof obligations, verification scope, claim traceability

This organization does not establish that every component has been independently verified or that all interfaces satisfy one end-to-end specification.

2.1 Canonical identity and lifecycle

Canonical event identity is intended to distinguish an event's identity from the number of times it is received, which matters for duplicate delivery and replay. P-System includes event identity, signed-event, lifecycle, and ingress components. A complete implementation-level guarantee requires a demonstrated correspondence between Python representations and abstract formal definitions, including identity encoding, signature validation, and lifecycle transitions. That correspondence remains open.

2.2 Causal ordering

P-System represents causal dependencies through graph- and closure-oriented components and includes execution-planning and state-derivation logic. The Lean model contains results concerning causal precedence, valid sequences, and topological ordering.

An event should not execute before its required causal predecessors. Deterministic selection among ready events can make execution reproducible, but does not by itself establish semantic correctness or replica convergence.

2.3 Conflict handling

The architecture includes conflict detection, conflict closure, conflict reconciliation, and conflict-aware execution. Its research objective is to distinguish incompatible histories from merely concurrent events and preserve relevant alternatives rather than treating arrival order as sufficient justification for choosing a winner. Theorems about conflict apply only within the definitions and assumptions of the formal model. Implementation-level guarantees require further correspondence evidence.

2.4 Reconciliation and local projection

Nodes can hold different subsets of events while disconnected. P-System includes reconciliation and state-derivation components. Three properties must be distinguished: knowledge preservation, deterministic projection for a given modeled knowledge state, and convergence after nodes receive equivalent relevant information. These properties are related but not interchangeable.

3. Formal Model and Verification Strategy

Project documentation identifies 16 principal formal modules, P01–P16, and 89 theorem declarations. The modules cover selected properties involving state transitions, causal validity, conflict semantics, knowledge consistency, reconciliation, duplicate ingress, and conditional retry behavior.

The documented Lean build succeeds, and the reviewed formal modules are documented as containing no `sorry`, `admit`, `axiom`, or `opaque`. This is not proof that every relevant property is formalized or that the Python implementation is fully verified.

3.1 Evidence categories

- **FORMAL:** a machine-checked theorem within the formal model.
- **REFINEMENT:** evidence that concrete Python behavior corresponds to the specification.
- **EMPIRICAL:** an executable test covering a specific scenario.
- **ASSUMPTION:** an external or environmental condition required by a claim.
- **OPEN:** a proof, refinement, or experiment that remains incomplete.

Tests and formal proofs answer different questions. Tests demonstrate behavior for executed scenarios; theorems establish propositions within a model and its assumptions.

3.2 Implementation refinement

A principal open obligation is to relate concrete implementation states to abstract formal states. The intended refinement condition requires each relevant implementation transition to correspond to an allowed specification transition while preserving a representation relation. The project documentation identifies this refinement as incomplete; Lean results therefore must not be interpreted as a proof of the complete Python application.

3.3 Security and liveness boundaries

Claims concerning authorization, secure transport, Byzantine behavior, or eventual reconnection require explicit assumptions. A theorem about an abstract signature relation is not a complete security analysis of a deployed protocol. Conditional liveness does not guarantee reconnection if the environment never restores connectivity.

4. Installation and Execution

The following commands document the project's setup and execution paths. The main entry point and dashboard commands were reported as working in the author's development environment.

4.1 Reported environment

- Kali GNU/Linux Rolling 2026.3
- Python 3.14.7
- Lean 4.34.1
- Lake 5.0.0

4.2 Installation

From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

For optional dashboard and benchmark dependencies:

```
python -m pip install -r requirements-optional.txt
```

For development and testing:

```
python -m pip install -r requirements-dev.txt
```

Exact dependency versions should be recorded for a fully reproducible archival release.

4.3 Launching P-System

Main entry point:

```
.venv/bin/python P-System.py
```

Direct dashboard entry point:

```
.venv/bin/python P-System-dashboard-rewrite.py
```

Both commands were reported as working in the author's development environment. Their portability to other environments has not been established by that observation alone.

4.4 Running tests

```
.venv/bin/python -m pytest -q
```

The recorded development run completed with 440 passing tests in 22.40 seconds. This supports the claim that the suite completed successfully for the tested revision and environment. It does not establish exhaustive coverage of all event sequences, concurrency schedules, network partitions, or adversarial inputs.

4.5 Building the formal model

```
cd formal/lean
lake build
```

The recorded build completed successfully with 31 jobs using Lean 4.34.1 and Lake 5.0.0. A successful build establishes that the configured formal source compiled; individual theorem statements and assumptions still require review.

4.6 Reproducibility record

The release is identified by Git tag `v0.1.0`, pointing to commit `ee8b054e5f3316e537a2f9a62b6d16fe18ada6cf`.

A reproduction report should record the exact source revision, hardware, operating system, language and dependency versions, commands and outputs, deviations from the documented environment, and raw benchmark results if experiments are performed. The reported test duration is an observation, not a general performance result.

5. Evaluation Methodology

The reported evaluation consists of an automated Python test run and a Lean 4 build. The Python suite passed 440 tests in 22.40 seconds. The Lean build completed successfully with 31 jobs.

The repository contains benchmark scripts, but this paper does not report validated comparative performance results. A future benchmark study should record the source revision, hardware, software versions, workload size,

event distribution, conflict rate, network conditions, repetitions, summary statistics, and raw results. Baselines must satisfy equivalent correctness requirements. No performance or scalability advantage is claimed.

6. Results

Evaluation item	Reported result	Interpretation
Python test suite	440 passed in 22.40 s	Evidence for tested scenarios
Lean build	Successful, 31 jobs	Configured formal source compiled
Formal modules	16 principal modules, P01–P16	Documented model organization
Theorem declarations	89 documented	Count reported by project documentation
Implementation refinement	Open	Complete correspondence not established
Comparative benchmarks	No validated results reported	Performance advantage unestablished
Mission-specific validation	Not established	Operational suitability unestablished

The results support further investigation of deterministic event processing and explicit conflict semantics. They do not establish complete distributed-system correctness, full implementation refinement, universal convergence, Byzantine fault tolerance, or superiority over existing systems.

7. Discussion

P-System's research value lies in making event semantics, conflict handling, and verification boundaries explicit. Concurrent events need not be incompatible, and arrival order should not automatically be treated as a semantic conflict-resolution policy. Dependencies, compatibility, evidence, and state projection must be defined together.

Deterministic processing is a local semantic property unless stronger assumptions are introduced. Convergence depends on event knowledge, reconciliation, and execution semantics. The formal model's evidential value depends on the quality of definitions, assumptions, and correspondence with the implementation.

8. Limitations and Threats to Validity

Incomplete implementation refinement: the Python implementation is not fully proved to correspond to the Lean model.

Test coverage: tests do not exhaust all event sequences, concurrency schedules, partitions, or adversarial inputs.

No comparative performance results: the paper reports no validated throughput, latency, resource-consumption, or scalability comparison.

Assumption sensitivity: network availability, retry behavior, authorization, and secure transport depend on environmental and implementation assumptions.

Independent review: results are development results, not independent replication or peer-reviewed evaluation.

Deployment readiness: suitability for safety-critical systems, including spacecraft flight control, has not been established. Space-mission event coordination remains a research hypothesis requiring separate validation.

9. Future Work

1. Complete and document correspondence between Python events, state transitions, node knowledge, conflict handling, and the Lean model.
2. Review theorem statements and assumptions cited as evidence.
3. Expand fault-injection tests for duplication, reordering, reconnection, partitions, and conflicting histories.

4. Run reproducible performance experiments with explicit baselines and publish raw measurements.
5. Evaluate constrained-resource and intermittent-connectivity behavior before making mission-related claims.
6. Obtain independent technical review and replication.
7. Verify startup and shutdown procedures from a clean installation.
8. Archive the exact source and research materials under a persistent identifier.

10. Conclusion

P-System is an experimental approach to distributed event processing with explicit causal dependencies, canonical event identity, conflict handling, reconciliation, and a separate Lean 4 formal model.

The reported development baseline includes 440 passing Python tests and a successful Lean build. These results support further investigation, but do not prove complete implementation refinement, universal distributed-system correctness, or performance superiority. The principal next step is to connect the formal specification to the implementation and complement the verification baseline with reproducible comparative experiments.

P-System should be regarded as a research artifact whose claims are bounded by available evidence: an explicit architecture, executable implementation, machine-checked formal model, and documented verification obligations.

References

[1] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. https://doi.org/10.1145/359545.359563

[2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” in *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, Lecture Notes in Computer Science, vol. 6976, pp. 386–400, 2011. https://doi.org/10.1007/978-3-642-24550-3_29

[3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, vol. 12699, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

Research Artifact

Project: P-System

Version: 0.1.0

Git tag: `v0.1.0`

Source commit: `ee8b054e5f3316e537a2f9a62b6d16fe18ada6cf`

Author: Paolo Amodio

The source repository URL and archival DOI should be added after the repository is public and an archival record has been created.